

NOMBRE DE LA UNIVERSIDAD

MAESTRÍA EN INGENIERÍA DE SOFTWARE

Desarrollo de Aplicaciones Empresariales

Sistema de Reserva de Canchas Deportivas

Arquitectura de Microfrontends y Microservicios

Proyecto Integrador

Autores

Pablo Jara Muñoz

pablojaramunoz@gmail.com

Miguel Lara

miguel.larasj@gmail.com

Sebastián Uyaguari

sebastianuyaguari@gmail.com

Adrián Espinoza

adrian.espinoza1292@gmail.com

Docente

Nombre del Docente

Ciudad, 25 de agosto de 2026

Resumen

Palabras clave:

Índice

Resumen	i
1. Introducción	1
1.1. Planteamiento del problema	1
1.2. Justificación	2
1.3. Estructura del documento	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Alcance funcional	3
3.1. Roles y permisos	3
3.2. Módulos y pantallas	3
3.3. Fuera de alcance	4
4. Arquitectura de la solución	4
4.1. Visión general	4
4.2. Notación	4
4.3. Vistas arquitectónicas	5
4.4. Seguridad y control de acceso	8
4.5. Registro de decisiones de arquitectura	8
5. Modelo de datos	9
5.1. Estrategia de persistencia	9
5.2. Diagrama entidad-relación	9
5.3. Diccionario de datos	9
5.4. Integridad entre bases de datos	10
5.5. Restricciones que implementan reglas de negocio	10

6. Reglas de negocio	10
6.1. Catálogo de reglas	10
6.2. Validación de solapamiento de horarios	11
6.3. Estados de una reserva	11
6.4. Traducción de reglas a respuestas HTTP	12
7. Resultados	12
7.1. Funcionalidades implementadas	12
7.2. Cumplimiento de los criterios de aceptación	12
7.3. Verificación de las reglas de negocio	13
7.4. Limitaciones	14
8. Conclusiones	14
8.1. Lecciones aprendidas	14
8.2. Trabajo futuro	14
Referencias	17
A. Scripts de base de datos	17
B. Colección de pruebas de la API	17
C. Capturas de la aplicación	17
D. Repositorio del proyecto	17

Índice de figuras

1.	Vista de contexto: el sistema y sus dos tipos de usuario.	5
2.	Vista de contenedores: microfrontends, gateway, microservicios y bases de datos.	7
5.	Modelo entidad-relación, con la frontera de cada base de datos.	9
3.	Vista de componentes de ms-reservas	15
4.	Vista de despliegue: contenedores Docker, redes y volumen.	16

Índice de tablas

1.	Matriz de roles y permisos.	3
2.	Módulos y pantallas del sistema.	4
3.	Composición de microfrontends.	6
4.	Composición de microservicios.	6
5.	Reparto de bases de datos.	9
6.	Entidad reservas	10
7.	Reglas de negocio y dónde se implementa cada una.	11
8.	Correspondencia entre violación de regla y respuesta HTTP.	12
9.	Cumplimiento de los criterios de aceptación.	13
10.	Comprobación de las reglas de negocio.	13

Pendientes

1. Introducción

En el presente informe de arquitectura de software se aborda el diseño de un sistema web para la gestión de reservas de canchas deportivas de tenis, básquet y pádel. Esta solución busca centralizar y facilitar la gestión de disponibilidad, el agendamiento y la cancelación de reservas, así como la gestión de canchas, horarios, usuarios y reportes relacionados con su uso.

El sistema se plantea bajo el enfoque de aplicaciones empresariales, considerando tanto la operación diaria del proceso de reservas como la necesidad de mantener una solución organizada, escalable y mantenible. Para ello, se propone una arquitectura basada en microservicios y microfrontends, con una separación clara de responsabilidades entre los módulos funcionales y técnicos.

En el frontend propone utilizar Module Federation para integrar módulos independientes, mientras que en el backend se considera el uso de Spring Boot para el desarrollo de microservicios y PostgreSQL para la persistencia de datos del sistema. Con esta base, cada módulo puede evolucionar de manera más controlada y desplegarse de forma autónoma según las necesidades del sistema.

1.1. Planteamiento del problema

La gestión de reservas de canchas deportivas presenta dificultades cuando se realiza mediante registros manuales o canales no integrados entre sí, como hojas de cálculo, calendarios, pizarras cuadrículadas. En este escenario, la consulta de horarios disponibles es confusa si no se mantiene un orden establecido, el registro o cancelación de reservas puede depender de validaciones manuales y la administración pierde visibilidad oportuna sobre el uso de las canchas y la demanda del servicio.

Esta situación incrementa el riesgo de inconsistencias, como el solapamiento de horarios, la falta de trazabilidad sobre quién realizó una reserva y bajo qué condiciones. Además, limita la capacidad de la organización para gestionar datos personales, aplicar reglas de negocio de manera uniforme y disponer de reportes que apoyen la toma de decisiones.

Frente a este contexto, surge la necesidad de contar con un sistema web centralizado que permita consultar disponibilidad, registrar reservas, gestionar usuarios y canchas, y generar información útil para la operación y el control administrativo.

1.2. Justificación

La solución se aborda con una arquitectura basada en microservicios y microfrontends debido a que el dominio del problema presenta responsabilidades diferenciadas, como la gestión de usuarios, reservas, canchas y reportes. Separar estos componentes facilita su desarrollo, mantenimiento y evolución, evitando concentrar toda la lógica en una sola aplicación de gran tamaño.

Desde el punto de vista del frontend, el uso de microfrontends permite organizar la interfaz por módulos funcionales y mantener cierto nivel de independencia entre ellos, lo que resulta conveniente para incorporar nuevas funcionalidades sin afectar por completo la aplicación principal. En el backend, los microservicios favorecen el aislamiento de reglas de negocio, la autonomía y una mejor alineación entre cada servicio y su responsabilidad dentro del sistema.

En conjunto, esta aproximación responde no solo a una decisión tecnológica, sino también a la necesidad de construir una solución más ordenada, extensible y adecuada para un escenario empresarial donde distintos procesos deben integrarse sin perder claridad ni control.

1.3. Estructura del documento

2. Objetivos

2.1. Objetivo general

Diseñar e implementar un sistema web para la reserva de canchas deportivas de tenis, básquet y pádel, que permita consultar disponibilidad, crear y cancelar reservas, gestionar canchas y usuarios, y visualizar reportes básicos, mediante una arquitectura basada en microfrontends y microservicios.

2.2. Objetivos específicos

- Implementar las funcionalidades principales del sistema, de manera que los usuarios puedan consultar disponibilidad, crear y cancelar reservas, mientras que los administradores puedan gestionar canchas, usuarios y reportes.
- Diseñar una arquitectura basada en microfrontends que permita dividir la interfaz en módulos funcionales independientes, integrados dentro de una misma solución web.

- Diseñar una arquitectura de microservicios que distribuya de forma clara las responsabilidades del sistema en los módulos de usuarios, canchas, reservas y reportes.
- Definir una estrategia de persistencia de datos que facilite la gestión de la información del sistema y mantenga la independencia de responsabilidades entre los servicios.
- Incorporar en la solución las reglas de negocio relacionadas con disponibilidad, solapamiento de reservas, cancelaciones, estados de reserva y permisos según el rol del usuario.
- Justificar las principales decisiones arquitectónicas y tecnológicas adoptadas en el proyecto en función de criterios como modularidad, mantenibilidad, escalabilidad y alineación con el alcance funcional definido.

3. Alcance funcional

3.1. Roles y permisos

Instrucción. Una matriz de funcionalidad por rol se lee de un vistazo y evita párrafos repetitivos. Marca con «Sí» y «No»; si algún permiso depende de una condición (cancelar solo las propias), dilo en la celda.

Tabla 1: Matriz de roles y permisos.

Funcionalidad	Usuario final	Administrador
[...]	[...]	[...]

3.2. Módulos y pantallas

Instrucción. Los módulos funcionales con sus pantallas. Conviene que esta tabla case con los microfrontends de la Tabla 3: si un módulo no corresponde a ningún remote, o al revés, es señal de que la descomposición no está clara.

Tabla 2: Módulos y pantallas del sistema.

Módulo	Pantalla	Descripción
[...]	[...]	[...]

3.3. Fuera de alcance

Instrucción. *Enumera lo excluido. No es una disculpa: delimitar el alcance es una decisión de diseño, y declararla evita que se lea como una carencia.*

[Funcionalidades excluidas y el motivo.]

4. Arquitectura de la solución

Instrucción. *Esta sección responde a una sola pregunta: por qué el sistema está partido así y no de otra manera. Describir los componentes es la parte fácil; lo que se evalúa es la justificación. Apóyate en las vistas para mostrar la estructura y en el registro de decisiones para explicarla.*

4.1. Visión general

[Dos o tres párrafos: qué estilo arquitectónico se aplica, en cuántas piezas se divide el sistema y qué principio gobierna el reparto.]

4.2. Notación

Los diagramas siguen el modelo C4 [1]: contexto, contenedores y componentes, más una vista de despliegue. Están generados desde una única descripción en Structurizr DSL (diagramas/workspace.dsl), de modo que las cuatro vistas se mantienen consistentes entre sí por construcción.

4.3. Vistas arquitectónicas

Instrucción. Una subsección por vista. En cada una: el diagrama, y debajo la prosa que explica lo que el diagrama no puede decir — por qué esa frontera, qué pasa si un elemento falla, qué se decidió a propósito. Un diagrama sin texto no comunica; un texto que solo enumera lo que ya se ve en el diagrama, tampoco.

4.3.1. V-01 Contexto del sistema



System Context View: Sistema de Reserva de Canchas
Nivel 1 — El sistema y sus usuarios.
Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 1: Vista de contexto: el sistema y sus dos tipos de usuario.

[Qué resuelve el sistema y para quién. Un párrafo.]

4.3.2. V-02 Contenedores

Es la vista principal del documento: contiene a la vez la composición de microfrontends y la de microservicios.

Frontend. [Los cuatro microfrontends, qué expone cada remote y cómo los integra el shell en tiempo de ejecución.]

Tabla 3: Composición de microfrontends.

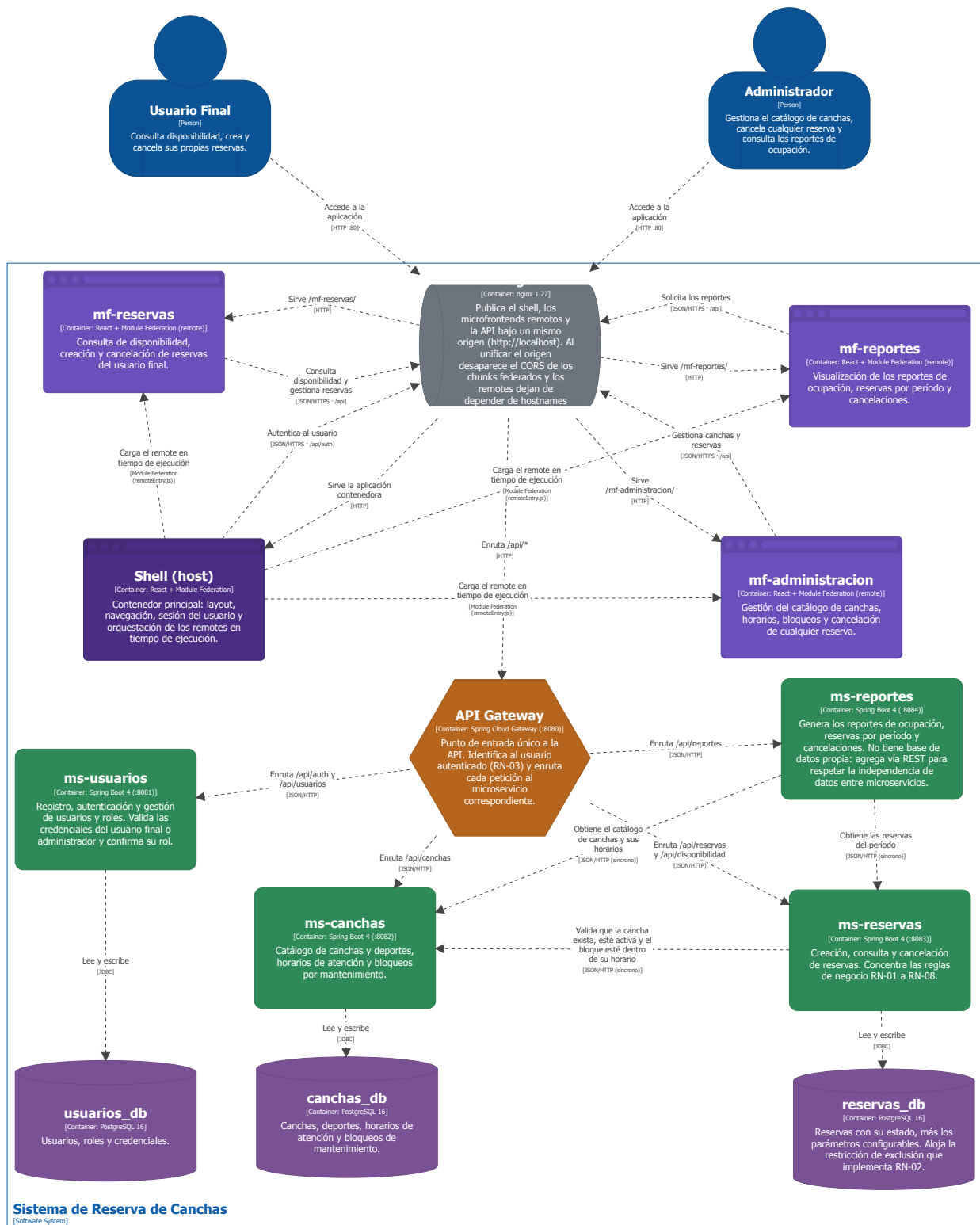
Microfrontend	Responsabilidad	Tipo
shell	[...]	Host
mf-reservas	[...]	Remote
mf-administracion	[...]	Remote
mf-reportes	[...]	Remote

Backend. [Los cuatro microservicios y el gateway. Señala que ms-reportes no tiene base de datos propia y por qué.]

Tabla 4: Composición de microservicios.

Microservicio	Responsabilidad	Base de datos
ms-usuarios	[...]	usuarios_db
ms-canchas	[...]	canchas_db
ms-reservas	[...]	reservas_db
ms-reportes	[...]	ninguna

Comunicación. [Qué servicio llama a cuál, con qué protocolo, y qué ocurre si el destino no responde.]



Container View: Sistema de Reserva de Canchas
Nivel 2 — Microfrontends, gateway, microservicios y bases de datos.
Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 2: Vista de contenedores: microfrontends, gateway, microservicios y bases de datos.

4.3.3. V-03 Componentes de ms-reservas

Se detalla este servicio y no otro porque es donde viven las reglas de negocio del sistema (Apartado 6).

[Recorrido de una petición de reserva a través de los componentes.]

4.3.4. V-04 Despliegue

[Cómo se materializa la arquitectura en contenedores: qué redes hay, qué aísla cada una y dónde persisten los datos. El detalle operativo va en el manual de despliegue (E5), no aquí.]

4.4. Seguridad y control de acceso

[Dónde se autentica el usuario, quién emite el token, quién lo valida y cómo llega la identidad a cada microservicio.]

4.5. Registro de decisiones de arquitectura

Instrucción. *Cada decisión relevante con su alternativa descartada. Una decisión sin alternativa no es una decisión: es una descripción. Este registro es lo que distingue un documento de arquitectura de un manual de usuario del propio sistema. Referencia las decisiones desde el texto con \adrrref{01}.*

ADR-01 [Título de la decisión]

Contexto. [Qué problema o fuerza obligó a decidir.]

Decisión. [Qué se decidió.]

Alternativa descartada. [Qué otra opción se consideró y por qué se descartó.]

Consecuencia. [Qué se gana y qué se paga por ello.]

Instrucción. *Candidatas a decisión en este proyecto, por si ayudan a arrancar: una base de datos por microservicio frente a un esquema compartido; la restricción de exclusión en PostgreSQL frente a validar el solapamiento en Java; el gateway como punto único frente a que cada microfrontend llame directamente a los servicios; ms-reportes agregando por REST frente a leer las tablas de otros servicios; el edge que unifica el origen frente a configurar CORS en cada remote.*

5. Modelo de datos

5.1. Estrategia de persistencia

Instrucción. *Explica el reparto: qué base pertenece a qué servicio y qué principio se sigue. Si se optó por una sola instancia de PostgreSQL con varias bases en lugar de varias instancias, dilo aquí y justifícalo — es una pregunta previsible en la defensa.*

[Una base por microservicio: qué implica y cómo se garantiza.]

Tabla 5: Reparto de bases de datos.

Base	Servicio dueño	Contenido
usuarios_db	ms-usuarios	[...]
canchas_db	ms-canchas	[...]
reservas_db	ms-reservas	[...]

5.2. Diagrama entidad-relación

Instrucción. *Un DER por base, o uno global con las fronteras entre bases marcadas. Lo segundo comunica mejor la independencia, que es justo lo que se evalúa.*

Falta figuras/modelo-datos.pdf
Exporta el diagrama desde Structurizr y déjalo en **figuras/** con ese nombre.

Figura 5: Modelo entidad-relación, con la frontera de cada base de datos.

5.3. Diccionario de datos

Instrucción. *Una tabla por entidad. No hace falta repetir todos los tipos SQL: basta el campo, su tipo lógico y qué significa. Los tipos exactos están en los scripts del anexo.*

Tabla 6: Entidad reservas.

Campo	Tipo	Descripción
[...]	[...]	[...]

5.4. Integridad entre bases de datos

Instrucción. *Es el apartado que más distingue un modelo de microservicios de un monolito con varias tablas. Explica qué referencias cruzan la frontera entre bases, por qué no pueden ser claves foráneas y dónde se valida entonces la integridad.*

[Referencias lógicas sin clave foránea: cuáles son, dónde se validan y qué ocurre si la validación falla.]

5.5. Restricciones que implementan reglas de negocio

Instrucción. *Aquí va la restricción de exclusión que impide el solapamiento. Es el punto técnico más fuerte del modelo de datos: explica por qué está en el motor y no en el código de la aplicación. El detalle de la regla como tal va en la Apartado 6.2; aquí solo el mecanismo de base de datos.*

[Restricciones e índices que sostienen las reglas y las consultas de disponibilidad.]

6. Reglas de negocio

6.1. Catálogo de reglas

Instrucción. *Una fila por regla, con la columna de implementación rellena de verdad: «ms-reservas, ReservaService» dice algo; «se valida en el backend» no. Esta tabla es la que se mira para puntuar este criterio.*

Tabla 7: Reglas de negocio y dónde se implementa cada una.

ID	Descripción	Implementación
RN-01	[Bloques horarios predefinidos]	[...]
RN-02	[No se permite solapamiento]	[...]
RN-03	[Permisos de cancelación]	[...]
RN-04	[No se cancelan reservas pasadas]	[...]
RN-05	[Cancelar libera el bloque]	[...]
RN-06	[Límite de reservas activas]	[...]
RN-07	[Solo el administrador gestiona canchas]	[...]
RN-08	[Estados de una reserva]	[...]

6.2. Validación de solapamiento de horarios

Instrucción. La regla que el documento de alcance destaca sobre las demás. Dedícale espacio: qué problema hay si dos personas reservan a la vez, dónde se resuelve y por qué ahí. Si la validación está en la base de datos y no en el servicio, explica el razonamiento: es un argumento de concurrencia, no de comodidad.

[Mecanismo elegido para **RN-02** y por qué resiste peticiones concurrentes sobre el mismo bloque.]

6.3. Estados de una reserva

[Los tres estados de **RN-08** y las transiciones permitidas entre ellos. Un diagrama de estados pequeño comunica esto mejor que un párrafo.]

6.4. Traducción de reglas a respuestas HTTP

Instrucción. *Qué código devuelve la API cuando se viola cada regla. Importa más de lo que parece: un 500 genérico ante una reserva duplicada es un defecto, y un 409 con un mensaje claro es lo que permite que la interfaz reaccione bien.*

Tabla 8: Correspondencia entre violación de regla y respuesta HTTP.

Regla	HTTP	Código de error y significado
[...]	[...]	[...]

7. Resultados

7.1. Funcionalidades implementadas

Instrucción. *Qué quedó operativo, con capturas de las pantallas principales. Dos o tres capturas bien elegidas valen más que ocho: la consulta de disponibilidad, la reserva y los reportes.*

[Resumen de lo implementado.]

7.2. Cumplimiento de los criterios de aceptación

Instrucción. *Los seis criterios del documento de alcance, uno por fila, con la evidencia que respalda cada «Sí». Es la tabla que un evaluador busca primero. Si alguno no se cumple, decláralo: un criterio marcado como cumplido que luego falla en la demo cuesta más que uno declarado pendiente.*

Tabla 9: Cumplimiento de los criterios de aceptación.

#	Criterio	Estado	Evidencia
CA-1	[Consultar, crear y cancelar reservas de las tres canchas]	[...]	[...]
CA-2	[El administrador gestiona el catálogo y cancela cualquier reserva]	[...]	[...]
CA-3	[El sistema impide reservar un bloque ocupado]	[...]	[...]
CA-4	[Los reportes muestran ocupación y reservas por período]	[...]	[...]
CA-5	[Shell y al menos dos remotes integrados con Module Federation]	[...]	[...]
CA-6	[Al menos dos microservicios independientes sobre PostgreSQL]	[...]	[...]

7.3. Verificación de las reglas de negocio

Instrucción. *Cómo se comprobó que cada regla funciona. No hace falta un capítulo de pruebas: una tabla con el escenario, el resultado esperado y el obtenido es suficiente y se lee en treinta segundos. La colección de peticiones va al anexo.*

Tabla 10: Comprobación de las reglas de negocio.

Regla	Escenario	Resultado
[...]	[...]	[...]

7.4. Limitaciones

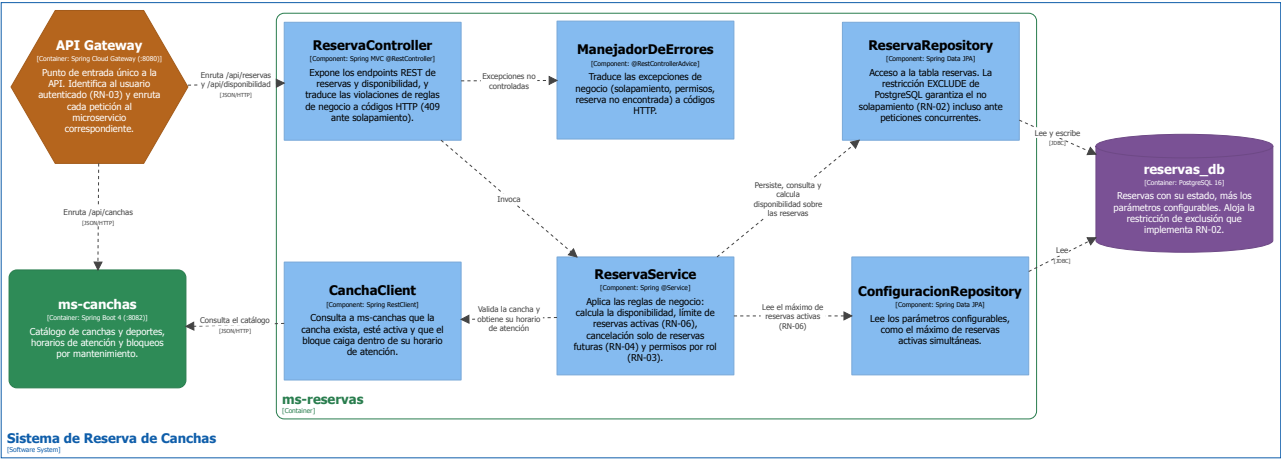
Instrucción. *Qué no se alcanzó y por qué. Declararlo suma: demuestra que se conoce el propio sistema. Omitir una limitación que el evaluador descubre en la demo resta el doble.*

[Limitaciones conocidas.]

8. Conclusiones

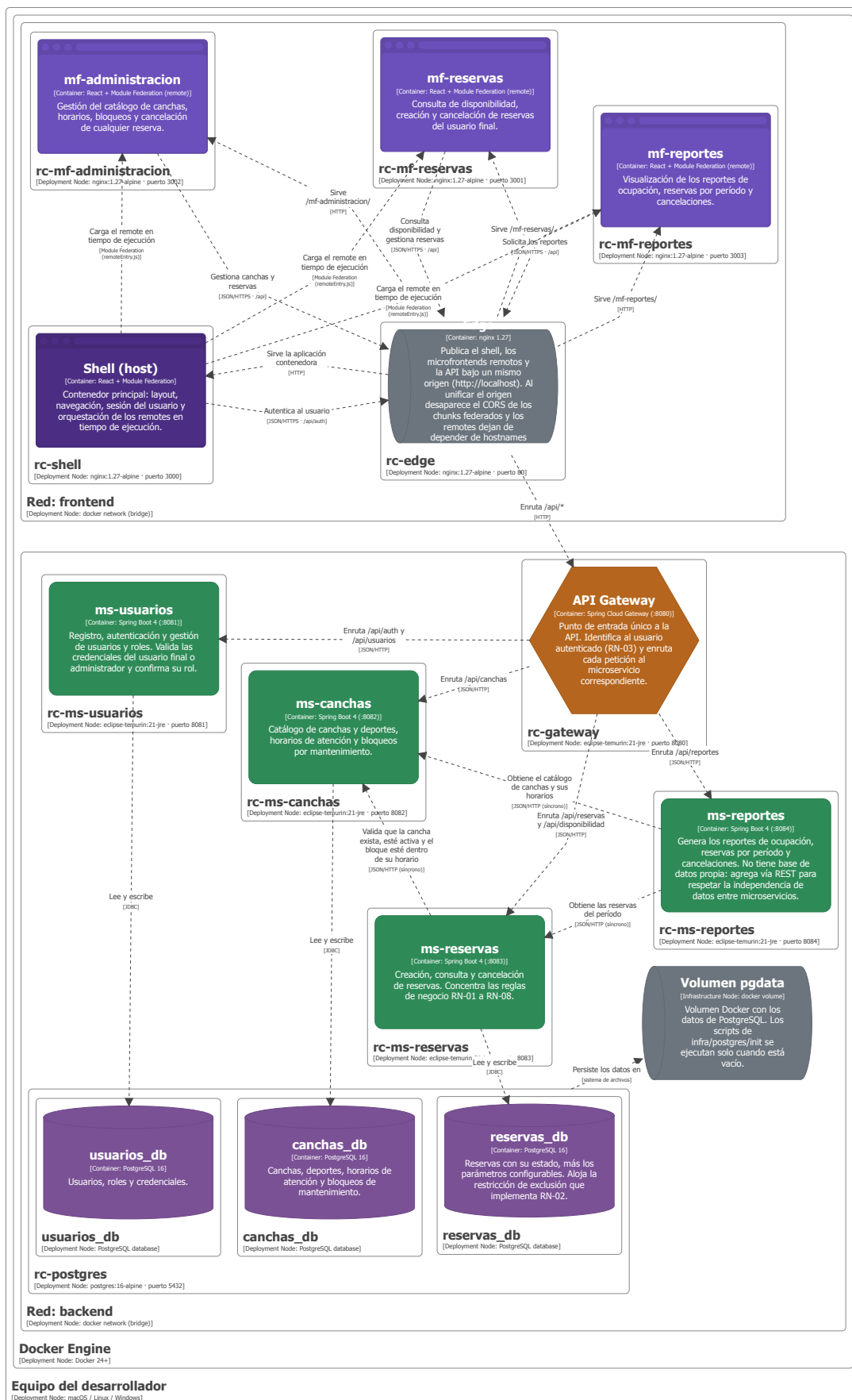
8.1. Lecciones aprendidas

8.2. Trabajo futuro



Component View: Sistema de Reserva de Canchas - ms-reservas
Nivel 3 — Interior de ms-reservas, donde se aplican las reglas de negocio.
Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 3: Vista de componentes de ms-reservas.



Deployment View: Sistema de Reserva de Canchas - Local (Docker Compose)
Materialización del sistema en contenedores Docker.
Sunday, August 23, 2026 at 10:04 PM Ecuador Time

Figura 4: Vista de despliegue: contenedores Docker, redes y volumen.

Referencias

- [1] Organización. «Título de la página o documentación,» visitado 1 de ene. de 2026.
dirección: <https://ejemplo.org/documentacion>

A. Scripts de base de datos

B. Colección de pruebas de la API

C. Capturas de la aplicación

D. Repositorio del proyecto